Open Design & Technology

Week 10

Client A :
"I have a rare tropical plant that needs exactly 4 hours of direct sunlight, but my apartment windows are small. I need a stand that rotates the plant towards the light and moves it away once it has had its daily quota."

Client B :
"In our hospital recovery wing, noise levels must stay low. I want an 'Indicator' light that is white normally, but turns blue and then red as the room gets louder, staying red for 10 seconds to 'shame' the noise-maker."

Systems Thinking

Approaching a Problem

Hardware Setup :

3 LEDs + Resistor + ESP32

Code Logic / Flow :

1.Blink 2 times @ blink interval of 0.5 seconds

2. Constantly ON for 2 Seconds

3. Right moving pattern with speed of 0.3 seconds in between each LED – Once

4. Blink 3 times @ blink interval of 1 second

A **function** is a reusable block of code that performs a specific task. It is defined once and can be called (executed) multiple times within a program.

It can take **inputs (parameters)**, process them, and optionally return an **output (result)**.

It helps in organizing code, improving readability, and avoiding repetition.

```
def function_name():
```

REPL Mode

Lost after reset

```
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :
    val_1 = button.value()

    if val_1 == 0:
        print("Monday")
        time.sleep(1)

    if val_1 == 0:
        print("Tuesday")
        time.sleep(1)

    if val_1 == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

Execution in RAM of ESP32

Runs **every time ESP32 boots**

```python
from machine import Pin

button = Pin(5,Pin.IN,Pin.PULL_UP)

while True :
    val_1 = button.value()

    if val_1 == 0:
        print("Monday")
        time.sleep(1)

    if val_1 == 0:
        print("Tuesday")
        time.sleep(1)

    if val_1 == 0:
        print("Wednesday")
        time.sleep(1)

    time.sleep(0.1)
```

main.py

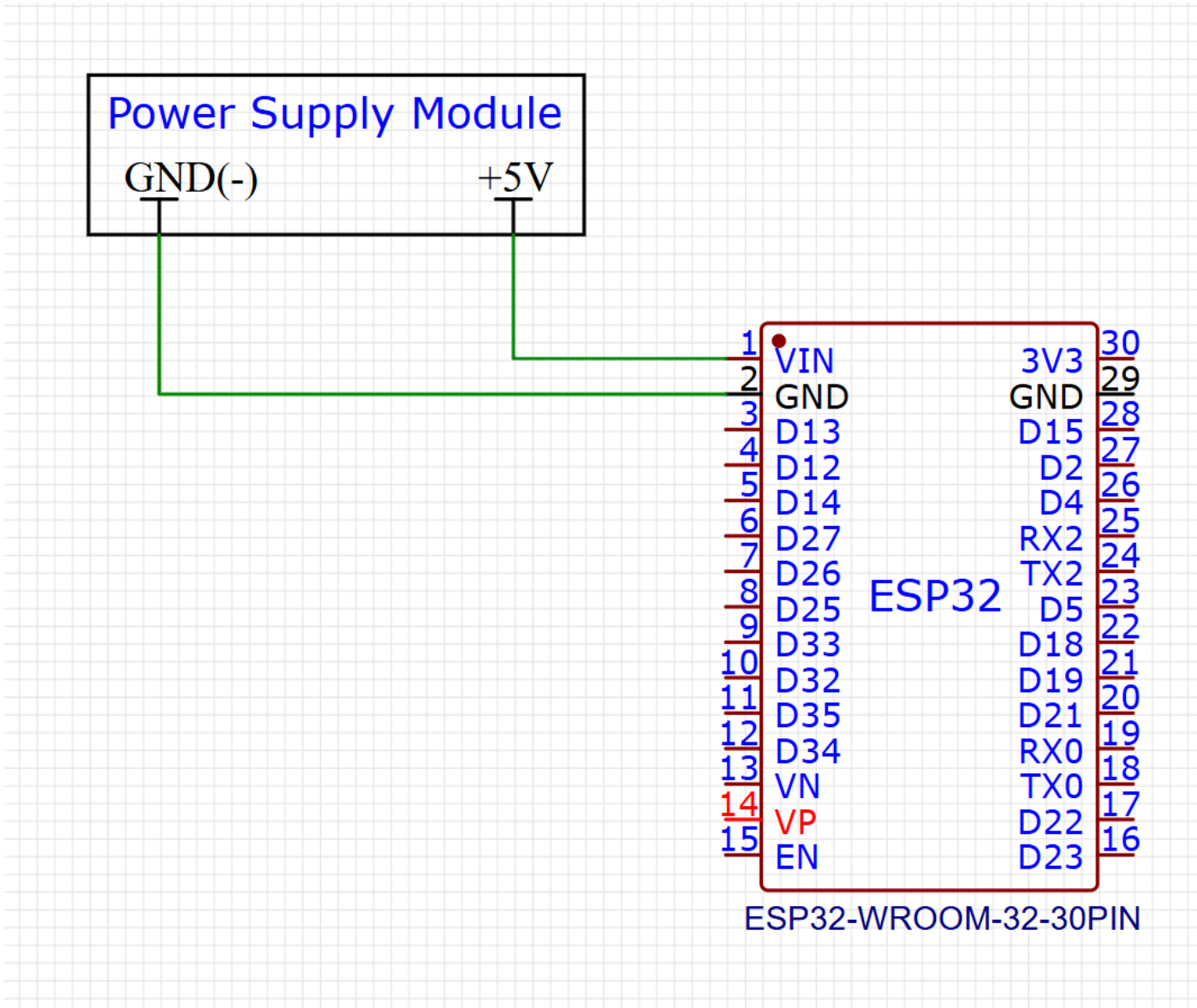
Stored in **FLASH** Memory of ESP32

Execution in RAM of ESP32

Save File to ESP32 (Flash Memory)

- Click **File** → **Save As**
- Choose: **“MicroPython device”**
- Save as: **main.py** (runs automatically on boot)
- File Gets Stored in Flash Memory

The saved file is now stored in ESP32’s non-volatile flash memory.


Power Supply Module
GND(-)
+5V
1 VIN
2 GND
3 D13
4 D12
5 D14
6 D27
7 D26
8 D25
9 D33
10 D32
11 D35
12 D34
13 VN
14 VP
15 EN
ESP32
30 3V3
29 GND
28 D15
27 D2
26 D4
25 RX2
24 TX2
23 D5
22 D18
21 D19
20 D21
19 RX0
18 TX0
17 D22
16 D23
ESP32-WROOM-32-30PIN

ESP32 is powered from an external power source and runs independently(without laptop)

main.py

Completely Digital

MIT App Inventor

Introduction To Bluetooth

BLE Vs Classic

BLE Communication There are always **2 Roles**

Server

(the device that holds data)

Example:

- **ESP32**
- Smart watch
- Heart rate sensor

The **server has the data.**

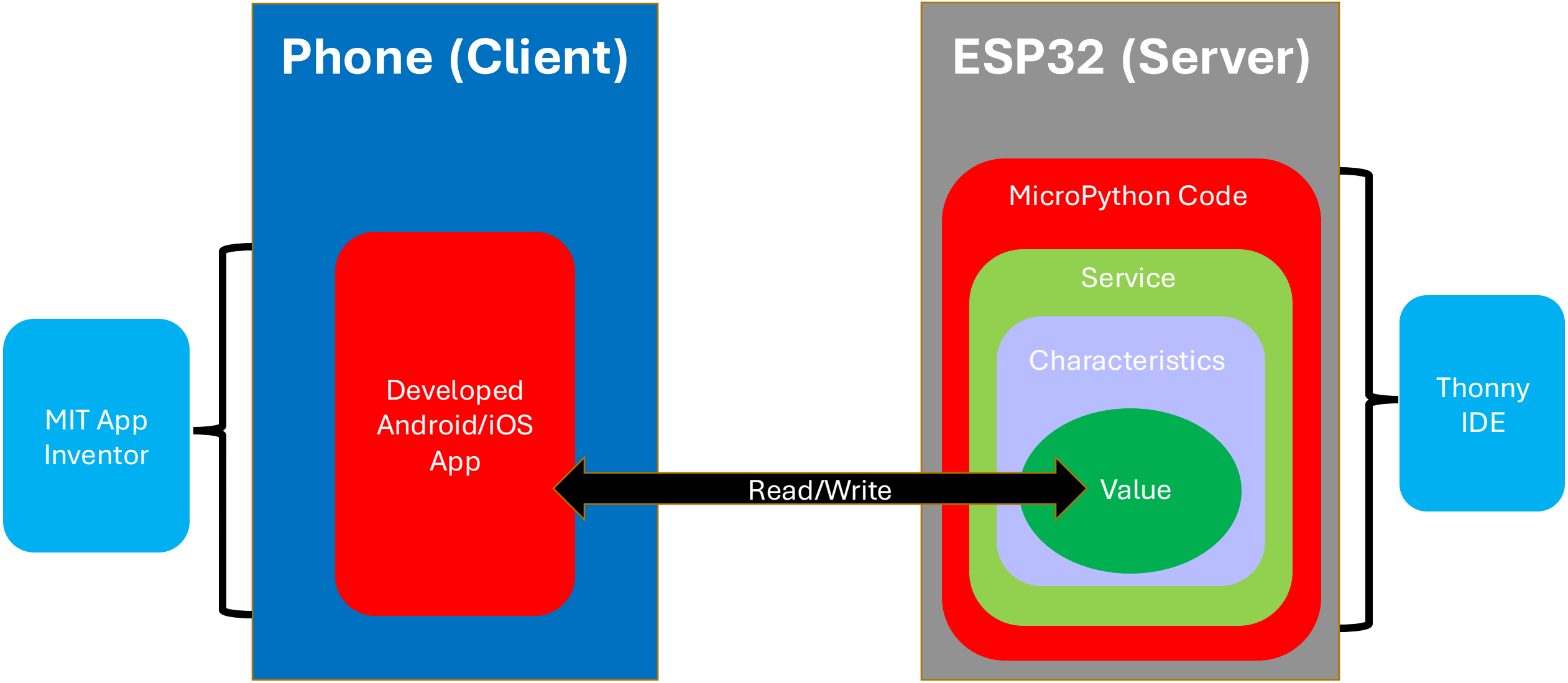
Client

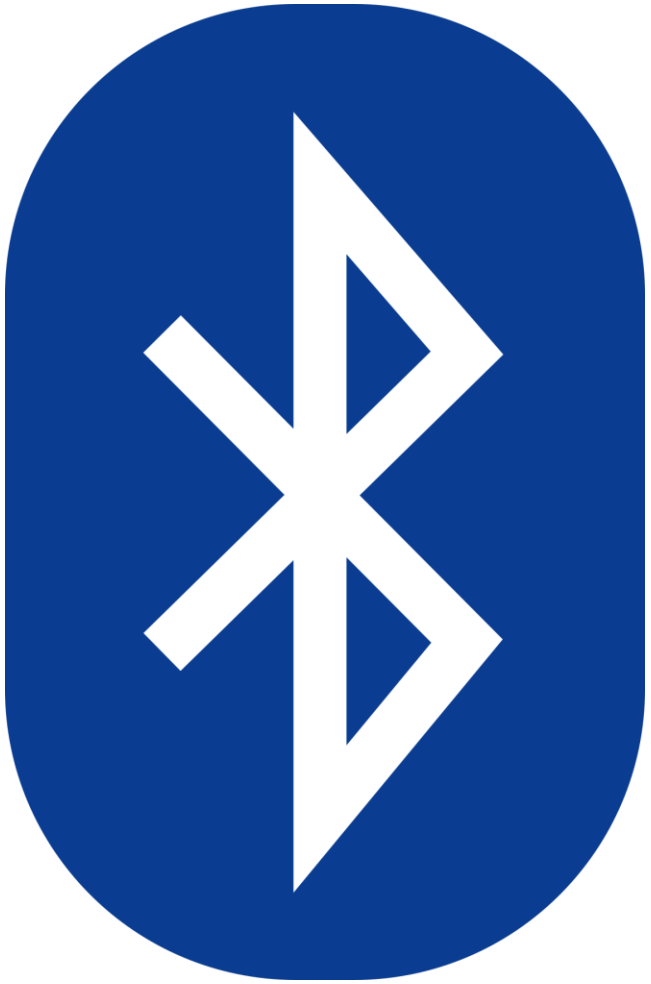
(the device that reads the data)

Example:

- Phone
- App made with **MIT App Inventor**
- Laptop

The **client asks for data** from the server.


Phone (Client)
ESP32 (Server)
MIT App Inventor
Developed Android/iOS App
Read/Write
MicroPython Code
Service
Characteristics
Value
Thonny IDE



Advertise → Device (ESP32) broadcasts its presence.

Scan → Phone/app searches for nearby BLE devices.

Connect → Phone establishes connection with ESP32.

Discover Services → Phone reads available services on ESP32.

Discover Characteristics → Phone finds data points inside services.

Write/Read → Phone sends data (write) or reads data from ESP32.

Process Data → ESP32 handles received data (e.g., control LED).

Notify (optional) → ESP32 sends data back to phone.

Disconnect → Connection is terminated.

Re-advertise → ESP32 becomes available again for new connections.

Sending data from Phone to ESP32

Using nRF Connect or LightBlue

Change the name of ESP32 in Micropython Code

Sending data from Phone to ESP32 to control LED

Using MIT App Inventor

Learning Objectives Week 10

- Insights about systems thinking
- Using Functions in Micropython
- Storing the code in flash memory of ESP32
- Using the breadboard power supply to power the ESP32
- Introduction to developing Android/iOS apps using MIT App Inventor
- Introduction to Bluetooth LE